

PROGRAMMER // 2026

プログラム説明資料

KuboEngine / Octopus

DirectX 12 × C++20。scene設計、CSVデータ駆動、OBB衝突、DX12リソース寿命管理を中心に構成。

C++20

DirectX 12

HLSL

ImGui

CSV

Telemetry

AUTHOR

TAKU OKUBO / オオクボ タク

ゲームプログラマー志望 / 日本工学院専門学校 ゲームクリエイター科 4年制

目次

作品概要から設計、実装、検証、今後の拡張までを順に説明。

01 制作情報

期間 / 人数 / 担当範囲 / 使用ライブラリ

03 作品概要

KuboEngine / Octopus のゲーム構成

05 ゲーム進行

GameplayFlowController

07 敵・衝突・負荷監視

空間分割、OBB、SoftCapTelemetry

09 データ駆動

CSV採用理由、strict parse、テスト

11 描画・リソース

FrameContext、SRV、cache

02 技術的な注力点

DX12、scene設計、CSV、衝突、検証

04 アーキテクチャ

GameSceneFactory と PlayScene

06 プレイヤー・武器

PlayerManager と WeaponController

08 レベルアップ

LevelUpChoiceService と重み調整

10 UI / Debug / 演出

HUD、MiniMap、DebugContext

12 まとめ

改善履歴、検証、今後の拡張

制作情報 / 担当範囲

個人制作として、ゲーム部分だけでなくエンジン基盤、データ、検証ツールまで担当。

制作期間

2025/07～開発中
延べ時間：約350～400時間

制作人数

個人制作
プログラム全般を担当

担当範囲

engine/、game/directxgame/、Resources/DirectXGame/data/、tools/、tests/ の実装・調整。

使用ライブラリ / 技術

C++20 / MSVC / Visual Studio
DirectX 12 / HLSL / Windows API
DirectXTex / Assimp
Dear ImGui / ImGuizmo / ImPlot / imgui-node-editor
FontAwesome icon font
CPU regression checks / scene transition stress / PIX preflight

制作で意識したこと

- ゲームを動かすだけでなく、scene遷移、resource寿命、調整データ、検証を自分で扱う。
- 固定値をCSVへ逃がし、武器・敵・UIを短い周期で調整できるようにする。
- debug UI、telemetry、CPU testで、実装の壊れ方を確認しやすくする。

技術的な注力ポイント

企業提出用として、実装力が伝わる3領域に紙面を寄せる。

DX12 リソース管理

- FrameContextでcommand allocator、upload arena、deferred release、fence valueをframe単位に管理。
- SrvManagerでdescriptor heapのallocate/freeとhigh-watermarkを記録。
- GameTextureCache / GameModelCacheで重複ロードと利用側APIを整理。

衝突・負荷監視

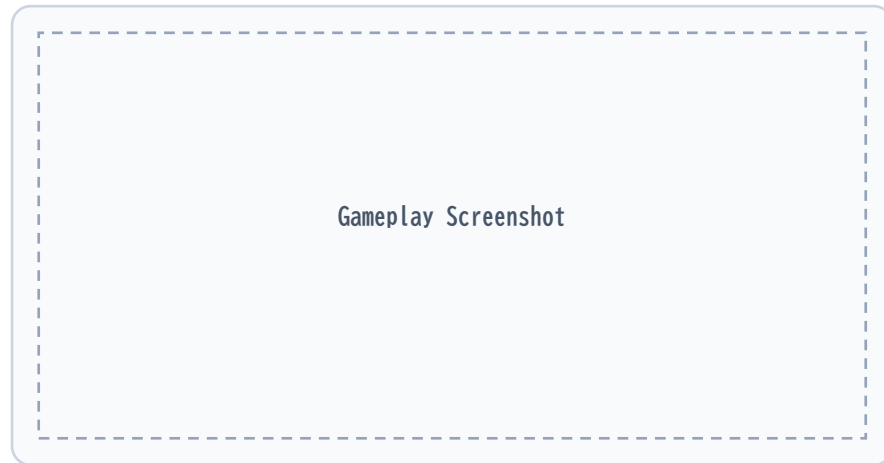
- EnemyCollisionContextでspatialMapとnearbyEnemiesを保持し、近傍候補を絞る。
- OBB narrow phaseで回転・スケールを含む当たり判定を扱う。
- SoftCapTelemetryで敵数、弾数、EXP Orb、prune数を確認する。

CSV データ駆動

- 武器、敵、level-up重み、UI配置、debug値をCSVへ分離。
- CSV採用理由は、表形式データをExcelとGit差分で確認しやすいため。
- CsvReaderのstrict parseとCPU testで不正値を検出する。

KuboEngine / Octopus

敵を倒してEXPを集め、レベルアップでビルドを強化していく3Dサバイバルシューティング。



Genre

3D Shooting / Survival Action

Role

1人制作 / コード全般

Engine

自作 DirectX 12 エンジン

Data

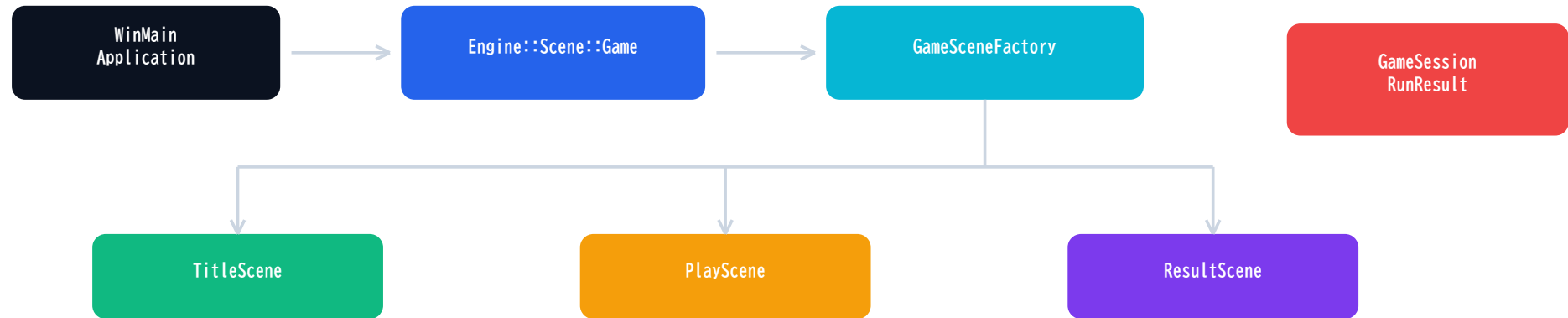
CSVによるバランス・UI調整

ゲームループ



シーン構成と責務分離

`main.cpp`は起動に集中し、scene生成はGameSceneFactoryへ渡す。PlaySceneはgameplayのcomposition root。



コード証拠: main.cpp

```

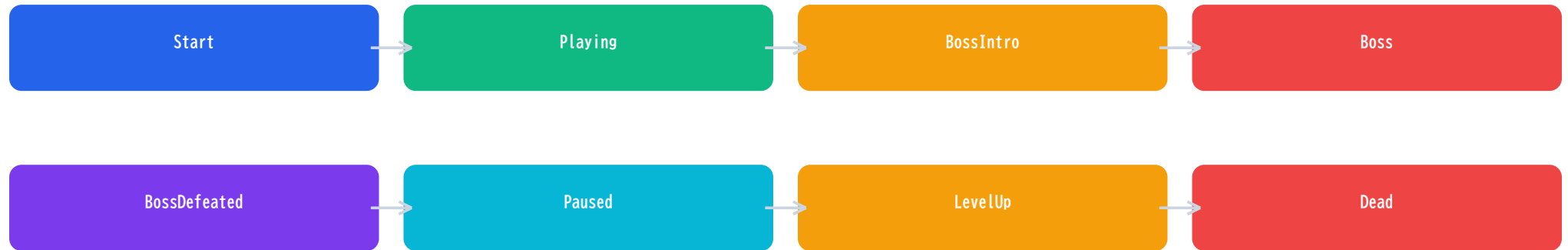
auto directXGame = std::make_unique<Engine::Scene::Game>(
    std::make_unique<DirectXGame::GameSceneFactory>(),
    DirectXGame::SceneId::kTitle);
std::unique_ptr<Engine::Base::Framework> game = std::move(directXGame);
game->Run();
  
```

設計意図

起動処理、共通Framework、ゲーム固有sceneの変更理由を分ける。新しいsceneや遷移を追加する場合も、main loopへ変更が広がりにくい。

GameplayFlowController

ゲーム中の状態を明示し、入力、world update、HUD、演出、scene遷移の切り替えを管理。



状態管理の工夫

- プレイ中、ポーズ中、レベルアップ中で、world update、HUD表示、cursor表示を切り替える。
- ボス登場、撃破、死亡演出をPresentation系クラスへ分離し、PlaySceneの肥大化を抑える。
- GameSessionがrun count、scene visit count、result情報を保持し、sceneをまたぐ状態を集約する。

成長と武器制御

PlayerManagerを窓口にし、進行状態と武器処理をPlayerProgression / PlayerWeaponControllerへ分割。



実装上の工夫

- `WeaponType` と `WeaponUpgradeData` により、武器種別と成長値を同じ流れで扱う。
- `weaponUpgradeSettings.csv` から弾数、間隔、半径、貫通、ダメージ補正を読み込む。
- 武器追加や成長値変更がPlayer本体へ直接広がらないよう、武器制御を専用クラスへ寄せた。

敵管理・OBB衝突・負荷監視

敵や弾が増える場面で、候補数、当たり判定、オブジェクト上限をまとめて扱う。

OBB Debug Screenshot

- `EnemyCollisionContext` が `spatialMap`, `activeEnemies`, `nearbyEnemies` を保持。
- `EnemyCollisionSystem` が Rebuild、Check、AreaDamage、Separation を担当。
- `SoftCapTelemetry` で enemy 数、kill 数、EXP Orb、弾数、prune 数を記録。

コード証拠: MakeCellKey

```
inline uint64_t MakeCellKey(int32_t cellX, int32_t cellZ)
{
    return (static_cast<uint64_t>(
        static_cast<uint32_t>(cellX)) << 32) |
        static_cast<uint32_t>(cellZ);
}
```

設計意図

負座標の cell も扱うため、`uint32\_t` 経由で `uint64\_t` に pack する。これにより、負値を左 shift する未定義動作を避ける。

LevelUpChoiceService

候補生成と適用処理を分離し、武器所持・レベル・HP状態に応じて候補を切り替える。

候補データ

```
enum class LevelUpUpgrade {
    Normal, Orbit, Drone, Lightning,
    Attack, MaxHp, MoveSpeed, Heal
};

struct LevelUpChoice {
    LevelUpUpgrade upgrade;
    std::string texturePath;
    std::string iconPath;
};
```

候補決定の流れ

Player状態

HP、所持武器、各武器Lv、上限到達を参照

Pool生成

未所持武器は解禁、所持済みは強化候補へ切替

Weight

`levelupWeights.csv` で出現比率を調整

Apply

`PlayerManager` のUpgrade/Add/Heal APIを呼ぶ

データ駆動型の調整基盤

CSVを採用した理由、弱点、対策を明記し、実装判断が伝わるページにする。

CSV

Excel編集、Git差分、表形式データと相性が良い

JSON

階層構造には強いが、武器Lv表では冗長

Binary

読み込み効率が良いが、手編集とdebugが難しい

C++直書き

型は強いが、調整ごとにbuildが必要

strict parse

```
const float result = std::stof(text, &parsedLength);
if (parsedLength != text.size() || !std::isfinite(result)) {
    throw std::invalid_argument("not a finite float");
}
```

CPU test

`1.0x`, `nan`, `12.0`, overflowなどを不正値として検出。CSVの編集しやすさと型の弱さを、parseとtestで補う。

主なCSV

- `playerStatus.csv`, `weaponUpgradeSettings.csv`, `enemyTypes.csv`, `enemySpawnSettings.csv`
- `levelupWeights.csv`, `ui\_layout\_title/hud/levelup/pause/result.csv`
- `resource\_manifest.csv`, `debug\_tuning.csv`, `soft\_cap\_telemetry.csv`

HUD・ミニマップ・調整環境

実行中の状態確認、当たり判定表示、telemetry確認を行うためのdebug環境。



確認サイクル



描画・リソース寿命管理

DirectX 12ではGPU参照中のresource寿命を明示する必要があるため、FrameContextとSrvManagerを中心に管理。

FrameContext

```
struct FrameContext
{
    ComPtr<ID3D12CommandAllocator> commandAllocator;
    ComPtr<ID3D12Resource> uploadArena;
    std::vector<ComPtr<ID3D12Resource>> deferredReleaseResources;
    uint64_t fenceValue = 0;
};
```

SrvManager

SRV heapを単独所有し、allocate/free、used count、high-watermark、usage recordを記録。

Cache

GameTextureCache、GameModelCache、GameAudioCacheで重複ロードと利用側APIを整理。

frame reuse



まとめ / 改善履歴 / 今後の拡張

実装した機能を並べるだけでなく、設計上どこを直し、何を検証したかを示す。

Scene安定性

GameSceneFactory、GameSession、RunResultへ整理し、scene遷移と結果共有の責務を明確化。

Resource寿命

FrameContext、deferred release、SrvManager high-watermarkでDX12 resourceの寿命を追える形にした。

CSV安全性

strict parseとCPU regression checksで、編集しやすさと壊れ方の検出を両立。

負荷監視

SoftCapTelemetryとscene transition stressで、弾・EXP Orb・descriptor使用量を数値で確認。

検証値

```
scene transition stress: PASS
title/game/result visits: 3 / 3 / 3
maxSrvUsed: 105
maxSrvHighWatermark: 105
```

今後の拡張

collision benchmark、PIX capture、gameplay/OBB/debug screenshotsを追加し、実装内容を画面と計測値でさらに説明できる状態にする。